



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Discover. Learn. Empower.

Experiment-2.1

Student Name: Gagnesh Kakkar

UID: 23BCS11196

Branch: B.E-C.S.E

Section/Group: 23KRG-2B

Semester: 5th

Date of Performance: 10/09/2025

Subject Name: DAA

Subject Code: 23CSH-301

1. **Aim:** Sort a given set of elements using the Quick sort method and determine the time required to sort the elements. Repeat the experiment for different values of n, the number of elements in the list to be sorted. The elements can be read from a file or can be generated using the random number generator.
2. **Objective:** To understand quick sort.
3. **Input/Apparatus Used:** Quick sort concept is used.
4. **Procedure/Algorithm: Pseudocode:**

Quick Sort is a Divide and Conquer algorithm. It picks an element as pivot and partitions the given array around the picked pivot. There are many different versions of quick Sort that pick pivot in different ways.

- Always pick first element as pivot.
- Always pick last element as pivot (implemented below)
- Pick a random element as pivot.
- Pick median as pivot.

The key process in quick Sort is partition (). Target of partition() is, given an array and an element x of array as pivot, put x at its correct position in sorted array and put all smaller elements (smaller than x) before x, and put all greater elements (greater than x) after x.

5. Code:

```
Code.cpp X
Code.cpp > ...
1  #include <iostream>
2  #include <stdlib.h>
3  #include <ctime>
4  using namespace std;
5
6  int partition(int arr[], int low, int high) {
7      int pivot = arr[high];
8      int i = low - 1;
9
10     for(int j = low; j < high; j++) {
11         if(arr[j] <= pivot) {
12             i++;
13             swap(arr[i], arr[j]);
14         }
15     }
16     swap(arr[i + 1], arr[high]);
17     return (i + 1);
18 }
19
20 void quickSort(int arr[], int low, int high) {
21     if(low < high) {
22         int pi = partition(arr, low, high);
23         quickSort(arr, low, pi - 1);
24         quickSort(arr, pi + 1, high);
25     }
26 }
27
28 int main() {
29     int n;
30     cout << "Enter number of elements (n): ";
31     cin >> n;
32
33     int *arr = new int[n];
34
35     srand(time(0));
36     for(int i = 0; i < n; i++) {
37         arr[i] = rand() % 10000;
38     }
39
40     clock_t start = clock();
41     quickSort(arr, 0, n - 1);
42     clock_t end = clock();
43
44     double timeTaken = double(end - start) / double(CLOCKS_PER_SEC);
45
46     cout << "Time taken to sort " << n << " elements: " << timeTaken << " seconds" << endl;
47
48     delete[] arr;
49     return 0;
50 }
```



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Discover. Learn. Empower.

6. Output:

```
PS D:\3rd Year\Semester-5\DAA_23BCS11196_KRG-2B\EXP-2.1> cd "d:\3rd Year\Semester-5\DAA_23BCS11196_KRG-2B\EXP-2.1\" ; if ($?) { g++ Code.cpp -o Code } ; if ($?) { .\Code }
● Enter number of elements (n): 1000
Time taken to sort 1000 elements: 0 seconds
● PS D:\3rd Year\Semester-5\DAA_23BCS11196_KRG-2B\EXP-2.1> cd "d:\3rd Year\Semester-5\DAA_23BCS11196_KRG-2B\EXP-2.1\" ; if ($?) { g++ Code.cpp -o Code } ; if ($?) { .\Code }
Enter number of elements (n): 200000
Time taken to sort 200000 elements: 0.062 seconds
● PS D:\3rd Year\Semester-5\DAA_23BCS11196_KRG-2B\EXP-2.1> cd "d:\3rd Year\Semester-5\DAA_23BCS11196_KRG-2B\EXP-2.1\" ; if ($?) { g++ Code.cpp -o Code } ; if ($?) { .\Code }
Enter number of elements (n): 5000000
Time taken to sort 5000000 elements: 13.458 seconds
```